

Universidad Politécnica de San Luis Potosí

LAB: FILE PATH TRAVERSAL SEQUENCES STRIPPED NON-RECURSIVELY

Presentado por:

Christian Alfredo Morales Meza - 184937

Asignatura: Fundamentos de Ciberseguridad

Profesor: Servando López Contreras

Fecha de entrega: 09/09/2026

Índice de contenido

Índice de contenido	2
Ficha del laboratorio	3
Objetivo del laboratorio	3
Explicación técnica de la vulnerabilidad	3
Procedimiento paso a paso.....	3
Evidencia del procedimiento.....	4
Evidencias obligatorias	8
Explicación del payload.....	10
Mitigación recomendada.....	11

Ficha del laboratorio

Laboratorio: LAB: FILE PATH TRAVERSAL SEQUENCES STRIPPED NON-RECURSIVELY

Nivel: Practitioner

Tipo de explotación: File Path Traversal (Evasión de Sanitización No Recursiva)

Objetivo del laboratorio

Exfiltrar el contenido del archivo sensible /etc/passwd aprovechando una vulnerabilidad de recorrido de directorios en el parámetro 'filename', evadiendo un filtro defensivo de reemplazo simple no recursivo.

Explicación técnica de la vulnerabilidad

La causa raíz de esta vulnerabilidad reside en una sanitización deficiente de la entrada proporcionada por el usuario en el parámetro de carga de archivos de la aplicación. Cuando un servidor web procesa solicitudes para la lectura de archivos estáticos (como imágenes de productos), suele concatenar una ruta base predefinida con el nombre del archivo recibido. Si la aplicación intenta defenderse contra el recorrido de directorios eliminando las secuencias de salto tradicional (../) mediante una simple operación de reemplazo de cadenas sin recursividad (por ejemplo, ejecutando una sola pasada de `string.replace("../", "")`), introduce una brecha crítica.

En la mecánica subyacente del sistema de archivos del sistema operativo, la secuencia '../' le indica al núcleo que debe subir un nivel en la jerarquía del árbol de directorios. Al utilizar un filtro no recursivo, la aplicación procesa la entrada una sola vez de izquierda a derecha. Si un atacante inyecta una secuencia anidada como '....//', el motor de sanitización identifica el patrón '../' dentro de la cadena y lo remueve; sin embargo, al eliminar esos caracteres, las porciones sobrantes en los extremos se colapsan automáticamente, reconstruyendo una secuencia '../' completamente válida justo antes de ser enviada a la API del sistema de archivos.

Dentro del marco OWASP Top 10:2021, esta falla se clasifica bajo la categoría A01:2021-Broken Access Control. En la triada CIA, el impacto principal se centra en la Confidencialidad, permitiendo a usuarios no autorizados leer archivos arbitrarios del sistema (código fuente, archivos de configuración con credenciales, claves criptográficas o cuentas del SO). En entornos de producción reales, si este vector se combina con funciones de escritura de archivos o inclusión local de archivos (LFI), el riesgo se eleva gravemente hacia la alteración de datos (Integridad) o la Ejecución Remota de Código (RCE), tomando el control total del servidor.

Procedimiento paso a paso

Iniciamos el análisis del entorno de pruebas configurando el proxy interactivo Burp Suite e interceptando el tráfico HTTP generado al explorar la tienda web vulnerable. Al cargar la página de inicio, se observan múltiples solicitudes GET dirigidas al endpoint '/image' para recuperar las imágenes de los productos expuestos en el catálogo.

Interceptamos una solicitud estándar que utiliza el parámetro de consulta 'filename=20.jpg' y la

transferimos al módulo Burp Repeater para realizar una evaluación analítica e iterativa de las respuestas de la aplicación.

Llevamos a cabo un primer intento de explotación inyectando la ruta absoluta '/etc/passwd' directamente en el parámetro 'filename'. El servidor responde con un código de estado HTTP 400 Bad Request acompañado del mensaje 'No such file', indicando que la aplicación no permite rutas absolutas directas o fuerza una ruta relativa base.

Posteriormente, probamos secuencias de salto estándar como '../../etc/passwd'. El servidor retorna nuevamente un error HTTP 400 Bad Request 'No such file'. Esto evidencia la presencia de un filtro defensivo del lado del servidor que intercepta y elimina las cadenas '../' del valor recibido.

Para verificar si la sanitización es lineal y no recursiva, diseñamos un payload de evasión con secuencias anidadas: '....//....//....//....//etc/passwd'. Al enviar esta petición, el filtro remueve la primera capa de patrones '../', colapsando los caracteres adyacentes y restaurando las secuencias de recorrido requeridas. El servidor procesa la ruta resultante '../../etc/passwd', responde con HTTP 200 OK y devuelve el contenido íntegro del archivo '/etc/passwd', logrando resolver con éxito el laboratorio.

Evidencia del procedimiento

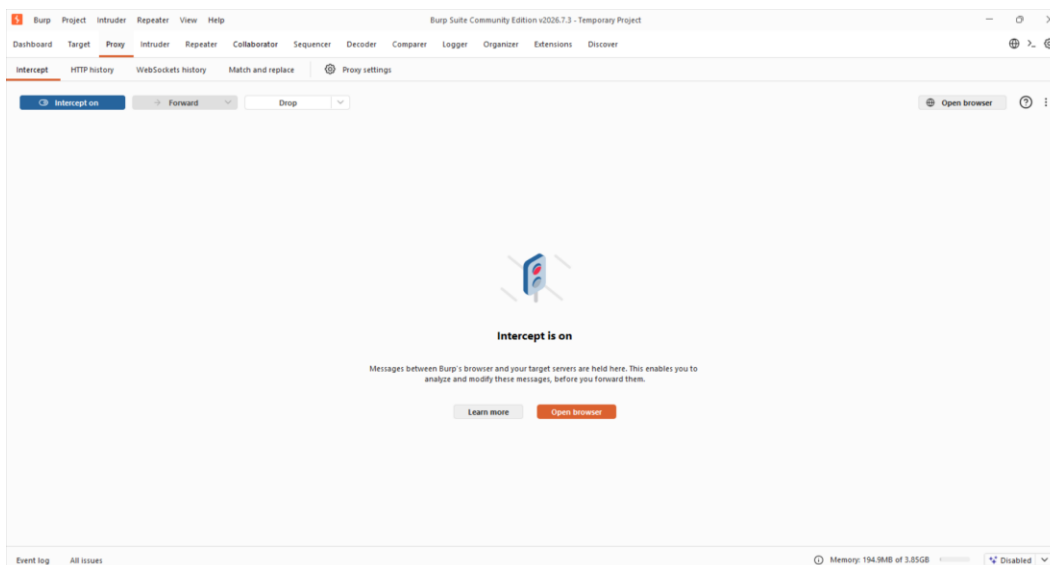


Figura 1. Configuración e inicio del Proxy de Burp Suite con la función de interceptación activada.

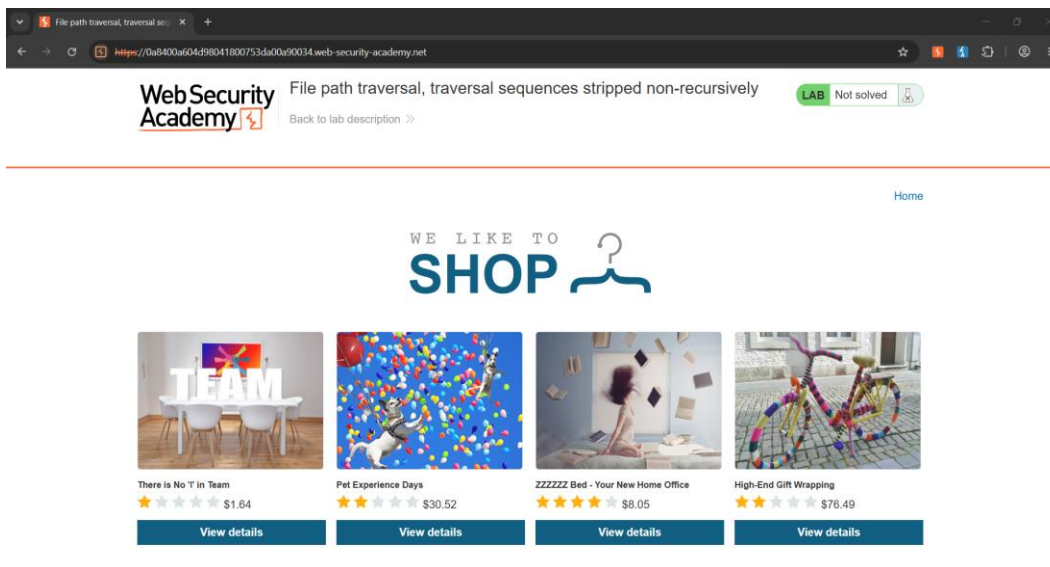


Figura 2. Navegación en la interfaz principal del laboratorio verificando la tienda de comercio electrónico.

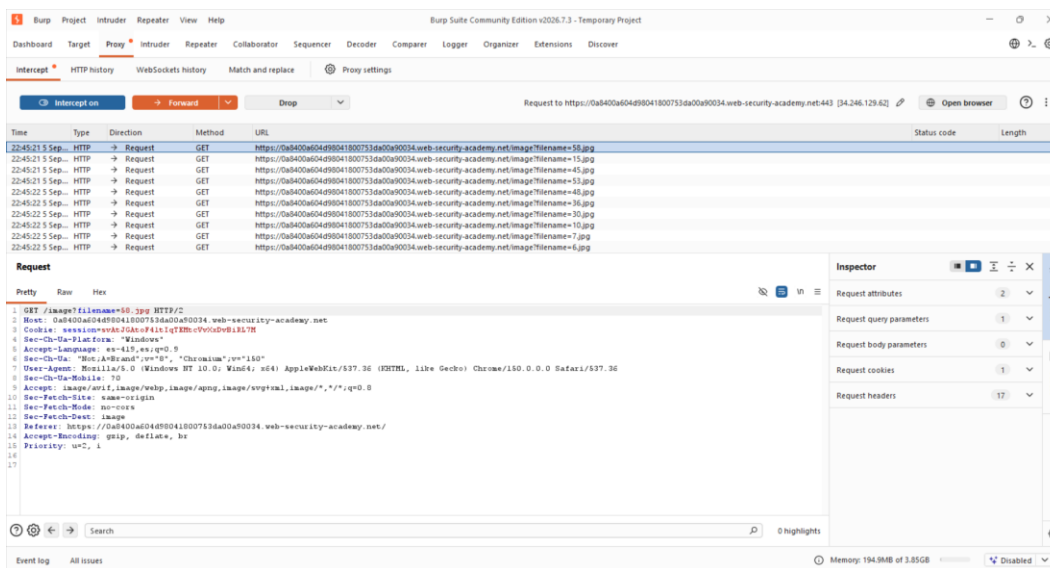


Figura 3. Intercepción de la petición GET dirigida al endpoint /image con el parámetro filename (por ejemplo filename=58.jpg).

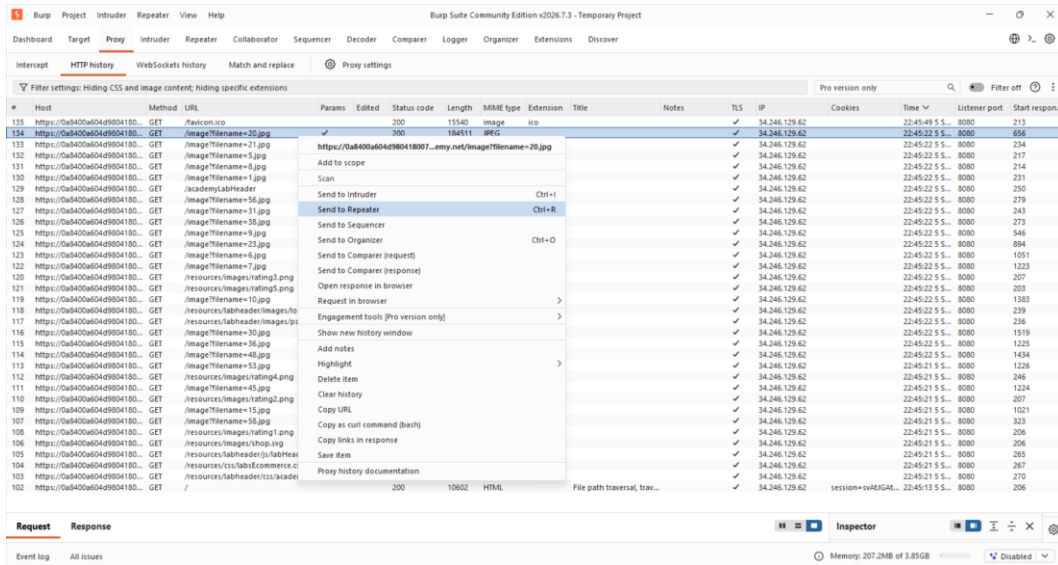


Figura 4. Transferencia de la solicitud interceptada hacia la herramienta Burp Repeater para la fase de pruebas en este ejercicio filename=20.jpg

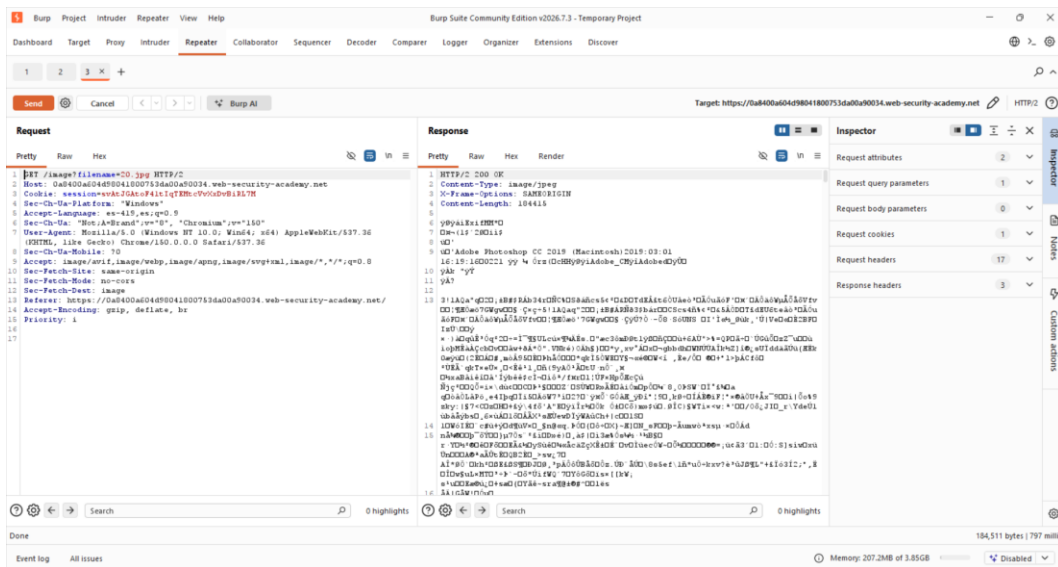


Figura 5. Envío de la petición original en Repeater recibiendo una respuesta HTTP 200 OK con el contenido binario de la imagen.

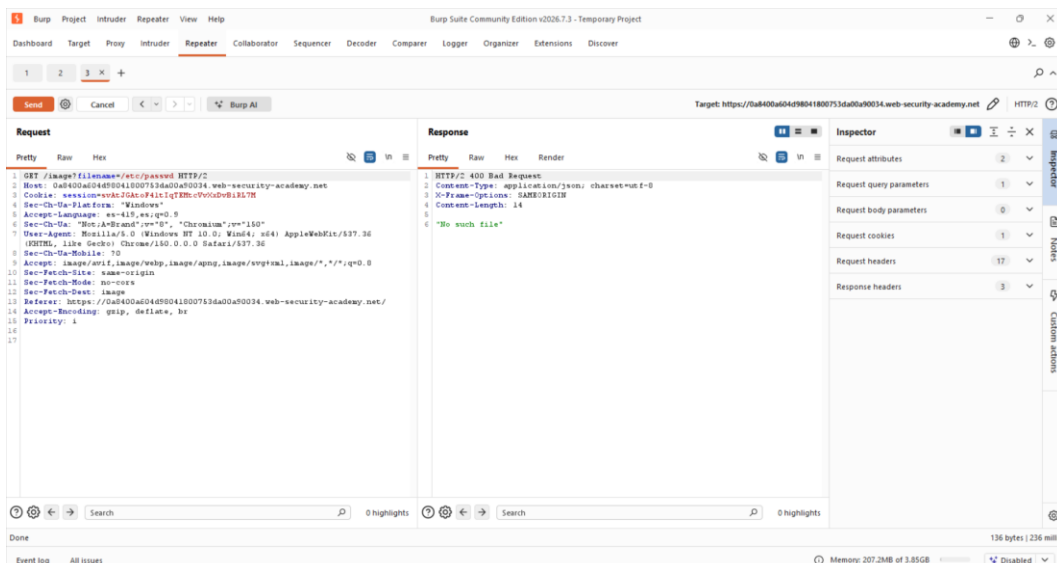


Figura 6. Intento de inyección utilizando la ruta absoluta /etc/passwd obteniendo un error HTTP 400 Bad Request.

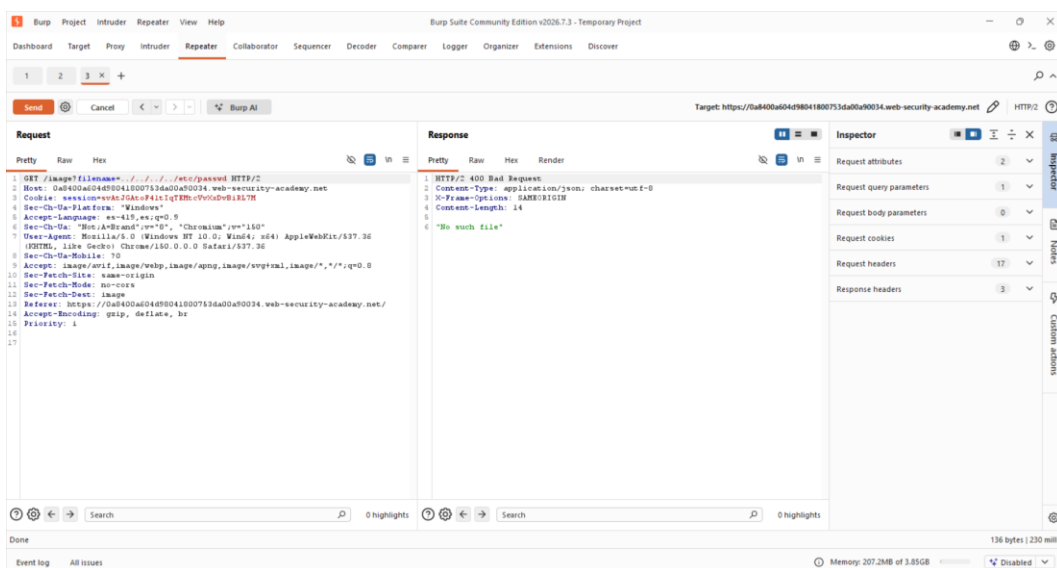


Figura 7. Prueba con secuencias de salto estándar .././../etc/passwd obteniendo como respuesta No such file.

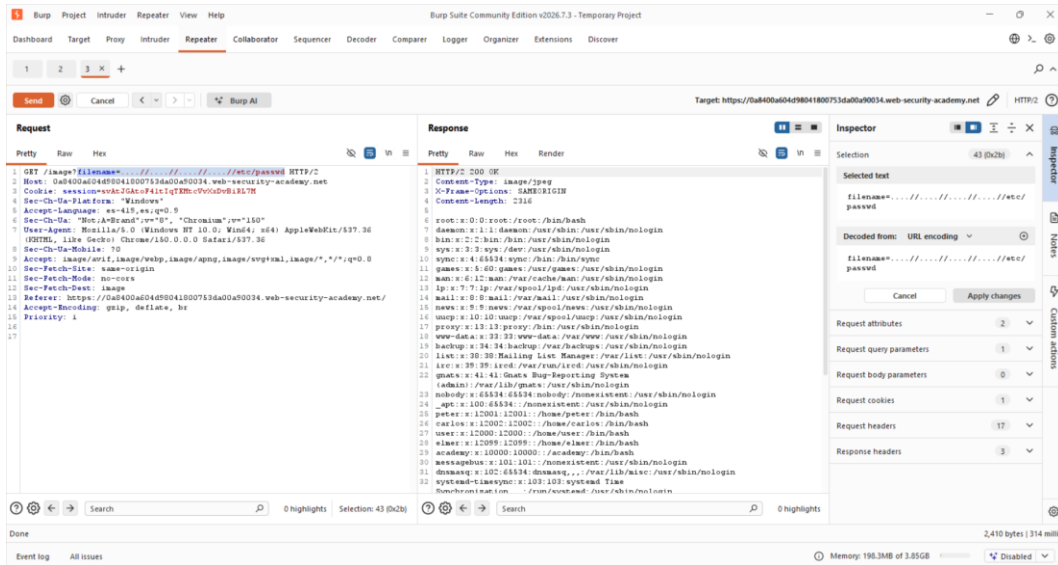


Figura 8. Inyección exitosa del payload con secuencias anidadas `../../../../../../../../etc/passwd` mostrando el archivo `/etc/passwd`.

Evidencias obligatorias

Request interceptado

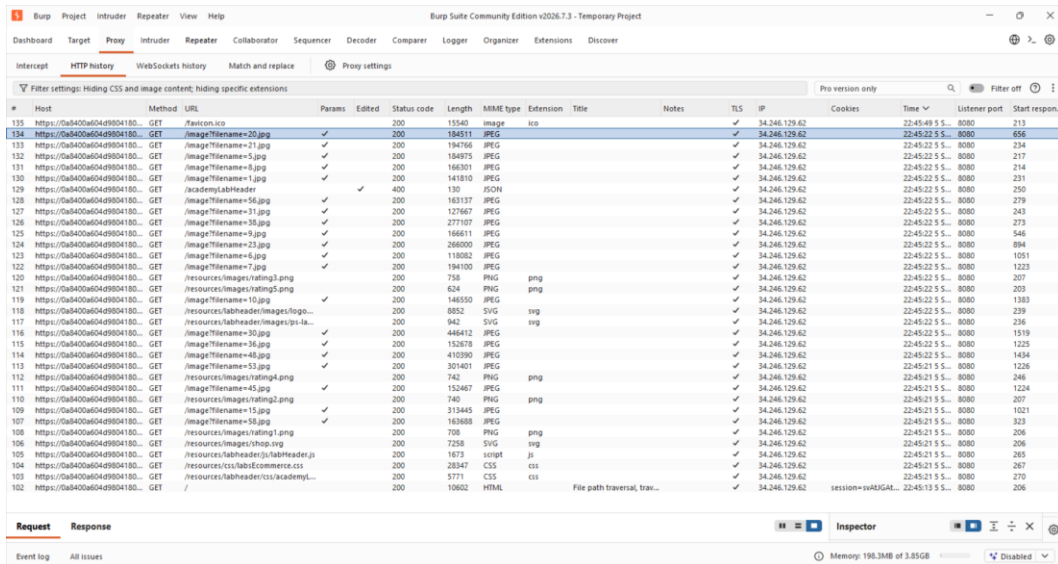


Figura 9. Request interceptado — Pestaña HTTP History de Burp Suite mostrando la interceptación de peticiones GET hacia el endpoint `/image` con el parámetro `filename` original.

Payload utilizado

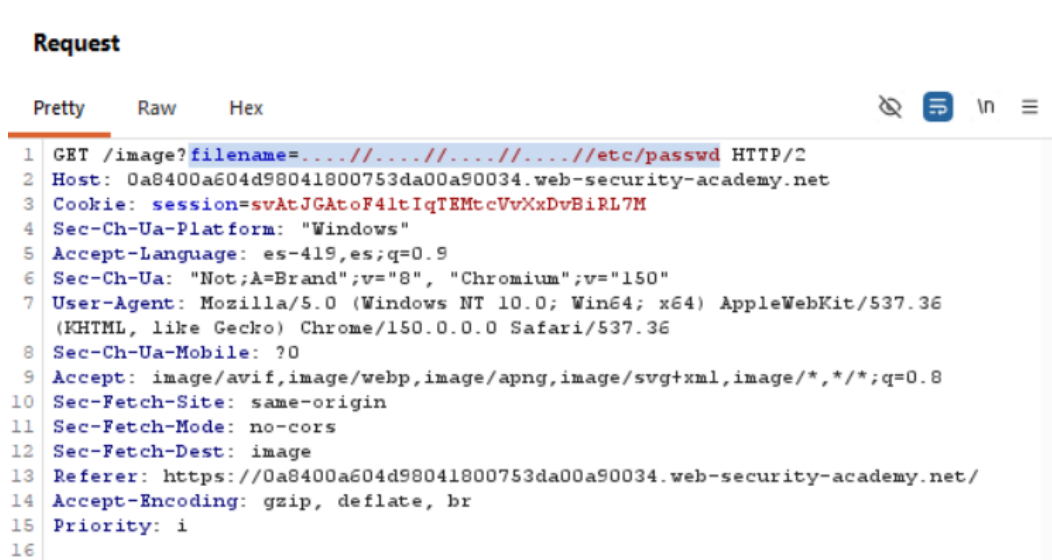


Figura 10. Payload utilizado — Inspector y editor de peticiones de Burp Suite mostrando el envío del payload anidado `../../../../../../../../etc/passwd` en el parámetro `filename`.

Respuesta vulnerable

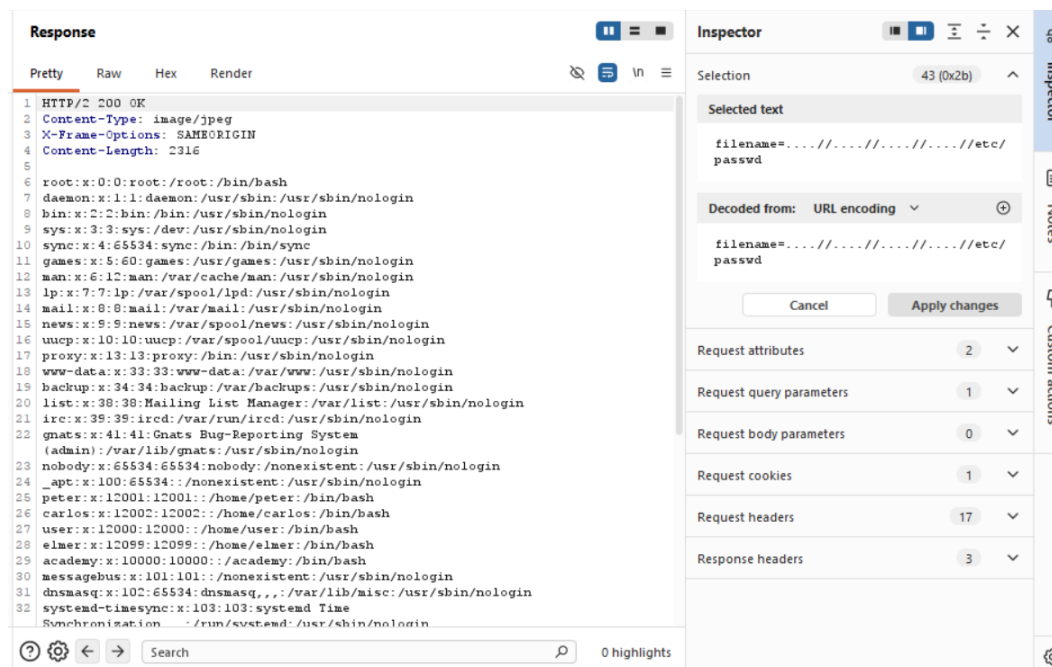


Figura 11. Respuesta vulnerable — Sección Response en Burp Suite desplegando un estado HTTP 200 OK y el contenido de texto plano extraído del archivo `/etc/passwd`.

Confirmación de “Lab Solved”

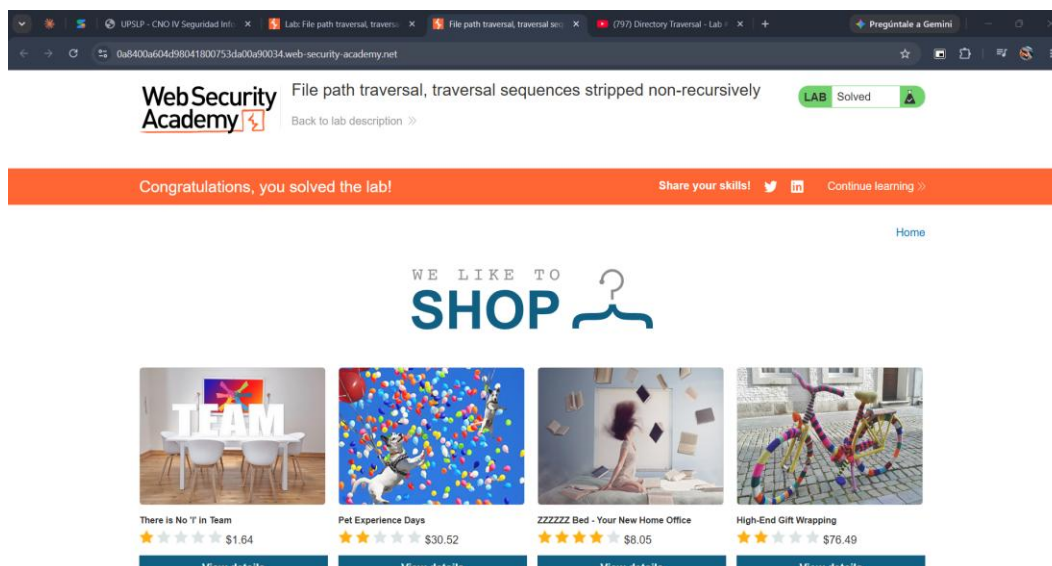


Figura 12. Confirmación de “Lab Solved” — Pantalla del navegador web con el banner de confirmación 'Congratulations, you solved the lab!' tras refrescar la aplicación.

Explicación del payload

El payload explotador utilizado es: 'filename=....//....//....//....//etc/passwd'.

Desglose sintáctico y lógico:

1. Parámetro objetivo: 'filename' en el parámetro GET del endpoint '/image'.
2. Mecanismo de evasión ('....//'): La cadena '....//' está compuesta por los caracteres '.' '.' '.' '.' '/' '/' '. Cuando el algoritmo no recursivo del backend busca la subcadena '../' para suprimirla, encuentra el patrón coincidente exactamente en la sección central de la cadena (caracteres 3 al 5: '..//').
3. Efecto de colapso: Al eliminar la secuencia interna '../', el primer par de puntos '..' sobrante se une con la barra final '/', reconstruyendo automáticamente una secuencia de salto de directorio pura '../'.
4. Profundidad del salto: Se replican cuatro bloques anidados '....//' seguidos de la ruta '/etc/passwd'. Tras pasar por la sanitización del backend, el payload se transforma internamente en '..//..//..//etc/passwd'.
5. Alteración del flujo: Las cuatro secuencias resultantes ascienden desde el directorio base donde residen las imágenes (por ejemplo, '/var/www/images/') hasta la raíz del sistema de archivos del sistema Linux ('/'), permitiendo la apertura directa y lectura del archivo de cuentas de usuarios del sistema '/etc/passwd'.

Mitigación recomendada

Para implementar una estrategia sólida de defensa en profundidad contra vulnerabilidades de Path Traversal, se recomiendan las siguientes medidas técnicas e integrales:

1. Evitar la entrada directa del usuario en el sistema de archivos: La práctica más segura consiste en sustituir los nombres de archivos directos por identificadores indirectos (p. ej., un ID numérico 'image_id=20') mapeados en una base de datos interna o lista blanca estricta.
2. Validación Canónica y Whitelisting: Si es indispensable recibir nombres de archivos, valide la entrada comparándola con una expresión regular estricta que únicamente permita caracteres alfanuméricos. Posteriormente, resuelva la ruta canónica del archivo utilizando las APIs nativas de la plataforma (por ejemplo, 'File.getCanonicalPath()' en Java o 'realpath()' en PHP/Python) y verifique formalmente mediante 'startsWith()' que la ruta canónica absoluta comience obligatoriamente en la ruta base autorizada (p. ej., '/var/www/images/').
3. Evitar filtros de sanitización manuales: No implemente filtros basados en la eliminación o reemplazo de cadenas (como 'replace("../", "")'), ya que suelen ser susceptibles a técnicas de evasión por anidación, doble codificación URL o caracteres nulos.
4. Principio de Menor Privilegio y Aislamiento: Configure la cuenta de servicio web (p. ej., 'www-data') con permisos mínimos estrictos de lectura únicamente en el directorio de recursos públicos. Utilice entornos chroot, contenedores o jaulas de aislamiento para limitar la visibilidad del proceso dentro del sistema operativo raíz.